

C Programming

Unit V — Functions & File Handling

Complete Formula Sheet • Revision Guide • Exam
Booster

SUBJECT

C Programming

UNIT

Unit V

TOPICS

Functions & File Handling

PAGES

~12 Pages

TABLE OF CONTENTS

- | | | | |
|----|-------------------------------|----|--------------------------------|
| 01 | Introduction to Functions | 02 | Declaration, Definition & Call |
| 03 | Function Types (4 Categories) | 04 | Arguments & Return Types |
| 05 | Call by Value vs Reference | 06 | Modifying Params via Pointers |
| 07 | Arrays as Function Parameters | 08 | Scope & Lifetime of Variables |
| 09 | Introduction to File Handling | 10 | File Operations & Functions |
| 11 | Reading & Writing Files | 12 | File Handling Programs |
| 13 | Comparison Tables | 14 | Key Terms & Definitions |
| 15 | Syntax Formula Box | 16 | Problem-Solving Patterns |
| 17 | Flashcards (30 Q&A) | 18 | Exam Tips & Mistakes |
| 19 | Quick Revision Sheet | | |

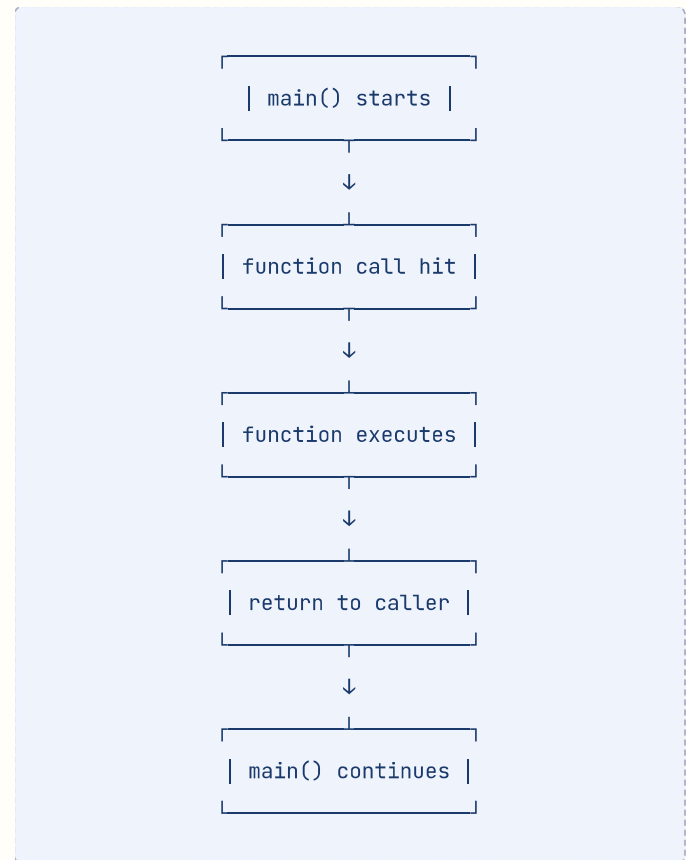
DEFINITION

A **function** is a self-contained, named block of code that performs a specific task. It can be called multiple times from anywhere in the program.

WHY FUNCTIONS?

- **Modularity** — Break program into logical units
- **Reusability** — Write once, use many times
- **Readability** — Clean, understandable code
- **Debugging** — Easier to isolate bugs
- **Team Work** — Multiple coders work in parallel

EXECUTION FLOW DIAGRAM



SYNTAX OVERVIEW

★ FUNCTION PROTOTYPE (DECLARATION)

```
return_type function_name(param_types);
```

★ FUNCTION DEFINITION

```
return_type function_name(params) { body }
```

★ FUNCTION CALL

```
function_name(arguments);
```

ROLE OF PROTOTYPE

- Tells compiler: name, return type, param types
- Must appear **before** first call
- Allows compiler to do type checking
- Ends with semicolon ;

COMPLETE EXAMPLE

```
// 1. Prototype (Declaration) int add(int, int);  
int main() { int result; result = add(3, 5); // 3.  
Call printf("%d", result); return 0; } // 2.  
Definition int add(int a, int b) { return a + b; }
```

FLOW DIAGRAM

```
Prototype → Compiler reads & remembers signature  
Call → Arguments matched with parameters  
Execution → Function body runs  
Return → Value sent back to caller
```


Type	Arguments	Return Value	Syntax Example	Use When
Type 1	✗ No	✗ No	void greet(void)	Just perform a task (print menu)
Type 2	✓ Yes	✗ No	void print(int x)	Act on data, don't need result back
Type 3	✗ No	✓ Yes	int getInput(void)	Generate/compute and return value
Type 4	✓ Yes	✓ Yes	int add(int a, int b)	Most common — compute & return

TYPE 1 — NO ARG, NO RETURN

```
void greet() {  
    printf("Hello!\n"); } // Call:  
greet();
```

TYPE 2 — WITH ARG, NO RETURN

```
void printSq(int n) {  
    printf("%d", n*n); } // Call:  
printSq(5);
```

TYPE 3 — NO ARG, RETURNS

```
int getNum() { int n;  
    scanf("%d", &n); return n; } //  
Call: x = getNum();
```

✓ TYPE 4 — WITH ARG, RETURNS (MOST COMMON)

```
int add(int a, int b) { return a + b; } // Call: result = add(3, 4); float avg(int x, int y) { return  
(x + y) / 2.0; } // Call: a = avg(10, 20);
```

FORMAL VS ACTUAL ARGUMENTS

Aspect	Formal Args	Actual Args
Location	Function definition	Function call
Role	Receive values	Provide values
Example	int add(int a, int b)	add(x, y)
Memory	Local to function	In calling scope

RETURN STATEMENT RULES

- Returns **one value** only
- Type must match declared return type
- void functions: use return; or omit
- After return, function execution stops
- Return value can be ignored by caller

MEMORY FLOW DURING FUNCTION CALL

STACK MEMORY

```
┌──────────────────┐ ← Stack grows ↓  
| main() frame |  
| x = 3, y = 5 |  
└──────────────────┘  
  
┌──────────────────┐ ← Created on call  
| add() frame |  
| a = 3 (copy) |  
| b = 5 (copy) |  
| return → 8 |  
└──────────────────┘ ← Destroyed on return
```

```
int add(int a, int b) { return a + b; // sends 8  
back } // In main: int x=3, y=5; int res = add(x,  
y); // res = 8
```




05 Call by Value vs Call by Reference

Feature	Call by Value	Call by Reference
What is passed	Copy of the value	Address (pointer) of variable
Original variable	NOT modified	CAN be modified
Memory usage	Extra copy created	No copy, uses same memory
Syntax (param)	void f(int x)	void f(int *x)
Syntax (call)	f(a)	f(&a)
Speed	Slower (for large data)	Faster (no copying)
Safety	Safer (original protected)	Less safe (original at risk)
Use case	When original shouldn't change	When original must be changed
Example	Swap doesn't work	Swap works correctly

CALL BY VALUE — SWAP FAILS

```
void swap(int a, int b) { int t = a; a = b; b = t;
// local only! } // After swap(x,y): x,y UNCHANGED
```

CALL BY REFERENCE — SWAP WORKS

```
void swap(int *a, int *b) { int t = *a; *a = *b;
*b = t; } // Call: swap(&x, &y); ✓
```

MEMORY DIAGRAM — CALL BY VALUE VS REFERENCE

Call by Value — swap(x, y)			Call by Reference — swap(&x, &y)		
main: x	10	addr:1000	main: x	10→20	addr:1000
main: y	20	addr:1004	main: y	20→10	addr:1004
func: a (copy)	10→20	addr:2000	func: *a	→1000	pointer
func: b (copy)	20→10	addr:2004	func: *b	→1004	pointer
x,y in main	UNCHANGED		x,y in main	SWAPPED ✓	



06 Modifying Parameters Using Pointers

CONCEPT

- Pass **address** of variable using &
- Receive as pointer with *
- Dereference pointer to read/write original
- Essential for: swap, modify arrays, multiple returns

```
void increment(int *p) { (*p)++; // modifies
original } int main() { int x = 5; increment(&x);
printf("%d", x); // 6 }
```

BEFORE / AFTER MEMORY STATE

BEFORE: increment(&x)		
x	5	addr:2000
p	2000	(pointer)
AFTER: (*p)++		
x	6	addr:2000
p	2000	still points

⚠ REMEMBER

`(*p)++` → increments value at p
`*p++` → increments the pointer itself!

HOW ARRAYS ARE PASSED

- Arrays are **passed by reference automatically**
- Only the **base address** (pointer) is passed
- No copy of the entire array is made
- Changes inside function affect original array

SYNTAX

✦ DECLARATION & CALL

```
void display(int arr[], int n);  
display(myArr, 5);
```

EXAMPLE

```
void printArr(int arr[], int n) { for(int i=0;  
i<n; i++) printf("%d ", arr[i]); } int main() {  
int a[] = {1,2,3,4,5}; printArr(a, 5); }
```

MEMORY DIAGRAM — ARRAY PASSING

```
main() frame:  
a[0]=1 [1002] | a[1]=2 [1006]  
a[2]=3 [1010] | a[3]=4 [1014]  
a[4]=5 [1018]  
  
printArr() frame:  
arr → points to 1002 (base addr)  
n = 5  
  
arr[0] = *(arr+0) → reads from 1002  
arr[1] = *(arr+1) → reads from 1006
```

💡 KEY NOTES

- `int arr[] = int *arr` in parameters
- Size not needed in function parameter
- Always pass size as separate argument
- Modifications in function affect original!

Variable Type	Declared Where	Scope	Lifetime	Default Value
Local	Inside a function/block	Within that block only	During function execution	Garbage (undefined)
Global	Outside all functions	Entire program	Entire program duration	0 (auto initialized)
Static Local	Inside function with static	Within that function	Entire program duration	0 (initialized once)
Register	Inside function with register	Within that block	During function execution	Garbage

LOCAL VS GLOBAL EXAMPLE

```
int g = 100; // global void show() { int x = 10;  
// local printf("%d %d", x, g); // x exists here  
only } int main() { show(); // printf(x); ERROR! x  
not here printf("%d", g); // OK }
```

STATIC VARIABLE EXAMPLE

```
void counter() { static int c = 0; // initialized  
ONCE only c++; printf("%d\n", c); } int main() {  
counter(); // prints 1 counter(); // prints 2  
counter(); // prints 3 }
```

💡 STATIC TIP

Static variables **retain** their value between function calls!

WHAT IS A FILE?

A **file** is a collection of related data stored permanently on a secondary storage device (disk). Data persists even after the program ends.

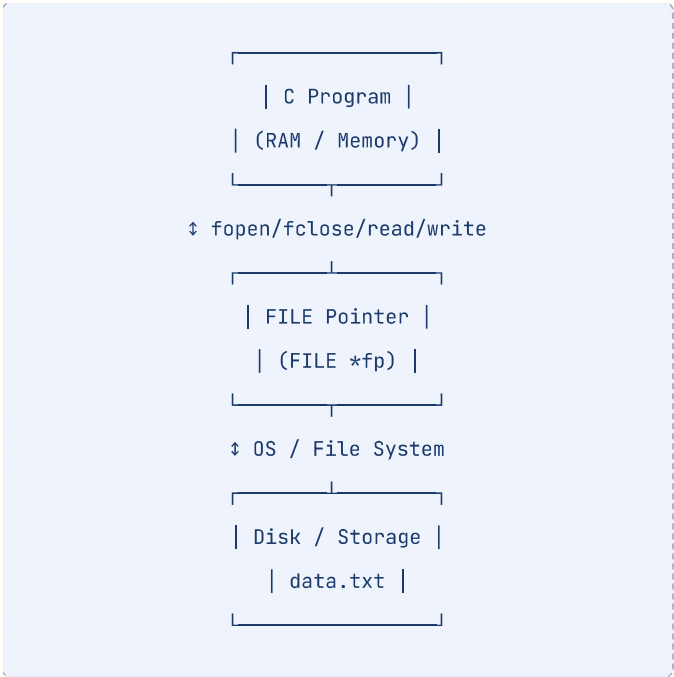
WHY FILE HANDLING?

- Data persists after program exits
- Store large amounts of data
- Share data between programs
- Input from / output to external files
- Avoid repeated data entry

FILE TYPES

Type	Data Format	Human Readable
Text File	ASCII characters	✔ Yes
Binary File	Binary (raw bytes)	✗ No

PROGRAM ↔ FILE ↔ STORAGE DIAGRAM



FILE MODES TABLE

Mode	Meaning	File Exists	File Not Exist
"r"	Read only	Opens it	Returns NULL
"w"	Write only	Overwrites	Creates new
"a"	Append	Appends to end	Creates new
"r+"	Read + Write	Opens it	Returns NULL
"w+"	Read + Write	Overwrites	Creates new
"a+"	Read + Append	Opens it	Creates new
"rb", "wb"	Binary modes	Same as above but binary	

CORE FILE FUNCTIONS

Function	Purpose	Returns
fopen()	Open a file	FILE* or NULL
fclose()	Close a file	0 or EOF
fprintf()	Write formatted to file	chars written
fscanf()	Read formatted from file	items read
fgetc()	Read one character	char or EOF
fputc()	Write one character	char or EOF
fgets()	Read a string/line	string or NULL
fputs()	Write a string	non-neg or EOF
fread()	Binary block read	items read
fwrite()	Binary block write	items written
feof()	Check end of file	non-zero if EOF
rewind()	Reset to file start	void

```
FILE *fp; // declare file pointer fp = fopen("data.txt", "r"); // open for reading if (fp == NULL) { //  
ALWAYS check! printf("File not found!\n"); return 1; } // ... perform operations ... fclose(fp); // ALWAYS  
close!
```

WRITING TO A FILE

```
FILE *fp; int x = 42; char name[] = "Alice"; fp =
fopen("out.txt", "w"); if(fp == NULL) { exit(1); }
fprintf(fp, "%d\n", x); fprintf(fp, "%s\n", name);
fputc('A', fp); fclose(fp);
```

READING FROM A FILE

```
FILE *fp; int x; char name[50]; fp =
fopen("out.txt", "r"); if(fp == NULL) { exit(1); }
fscanf(fp, "%d", &x); fscanf(fp, "%s", name);
printf("%d %s\n", x, name); fclose(fp);
```

READ FILE CHARACTER BY CHARACTER

```
FILE *fp; char ch; fp = fopen("test.txt", "r");
while((ch = fgetc(fp)) != EOF) { putchar(ch); //
print each char } fclose(fp);
```

READ FILE LINE BY LINE

```
FILE *fp; char line[100]; fp = fopen("test.txt",
"r"); while(fgets(line, 100, fp) != NULL) {
printf("%s", line); } fclose(fp);
```

COPY A FILE

```
FILE *src, *dst; char ch; src =
fopen("a.txt", "r"); dst =
fopen("b.txt", "w");
while((ch=fgetc(src))!=EOF)
fputc(ch, dst); fclose(src);
fclose(dst);
```

COUNT CHARACTERS

```
FILE *fp; char ch; int count =
0; fp = fopen("f.txt", "r");
while((ch=fgetc(fp))!=EOF)
count++; printf("Count: %d",
count); fclose(fp);
```

APPEND DATA

```
FILE *fp; char data[50]; fp =
fopen("f.txt", "a");
printf("Enter text: ");
fgets(data, 50, stdin);
fputs(data, fp); fclose(fp);
```

⚠ CRITICAL FILE HANDLING RULES

- Always check if fopen() returns NULL
- Always call fclose() when done
- Use "rb"/"wb" for binary files
- Don't read past EOF (check with feof())
- Use "a" mode to add without deleting old data
- "w" mode deletes existing file content!



FUNCTION VS PROCEDURE

Aspect	Function (C)	Procedure (Conceptual)
Returns value	✔ Yes (return type)	✘ No (void)
In C	int sum()	void show()
Purpose	Compute & return result	Perform an action

LOCAL VS GLOBAL VARIABLES

Feature	Local Variable	Global Variable
Declaration	Inside a function	Outside all functions
Scope	Within its block	Entire program
Lifetime	Function execution only	Entire program
Default value	Garbage (undefined)	0 (auto-initialized)
Memory	Stack	Global/BSS segment
Reuse risk	Safe (isolated)	Risk of unintended changes

TEXT FILE VS BINARY FILE

Feature	Text File	Binary File
Storage format	ASCII characters	Raw binary (bits/bytes)
Human readable	✔ Yes (in Notepad)	✘ No (garbage chars)
Mode	"r", "w", "a"	"rb", "wb", "ab"
Functions	fprintf, fscanf, fgets	fread, fwrite
Newline	Translated (\n ↔ \r\n)	No translation
Speed	Slower (conversion)	Faster (direct)
Use case	Config files, logs	Images, audio, struct data

FPRINTF VS FWRITE

Aspect	fprintf	fwrite
Type	Text	Binary
Format	Formatted string	Raw bytes
Readable	✔ Yes	✘ No
Speed	Slower	Faster

FSCANF VS FREAD

Aspect	fscanf	fread
Type	Text	Binary
Reads	Formatted data	Raw bytes/blocks
Use with	fprintf	fwrite

Syntax	fprintf(fp, "%d", x)	fwrite(&x, sizeof(x), 1, fp)
--------	----------------------	------------------------------

Syntax	fscanf(fp, "%d", &x)	fread(&x, sizeof(x), 1, fp)
--------	----------------------	-----------------------------

Function	Named block of reusable code that performs a task	Parameter	Variable in function definition that receives passed value
Argument	Actual value passed during a function call	Return Value	Value sent back from function to caller
Prototype	Function declaration before definition; tells compiler its signature	void	Return/parameter type meaning nothing/no value
Call by Value	Passing copy of variable; original not affected	Call by Reference	Passing address of variable; original can be modified
Pointer	Variable that stores memory address of another variable	Dereference (*)	Accessing value at the address held by a pointer
Address-of (&)	Operator that returns memory address of a variable	Local Variable	Variable declared inside a function; limited scope/lifetime
Global Variable	Variable declared outside all functions; program-wide scope	Static Variable	Retains value between function calls; initialized once
Scope	Region of code where a variable is visible/accessible	Lifetime	Duration for which a variable exists in memory
Stack	Memory region used for local variables and function frames	Recursion	Function calling itself; needs base case to terminate
File	Collection of related data stored permanently on disk	FILE *	Pointer type to a file structure; used in all file operations
fopen()	Opens a file; returns FILE pointer or NULL on failure	fclose()	Closes an open file; flushes buffer to disk
fprintf()	Writes formatted text to a file (like printf to file)	fscanf()	Reads formatted data from file (like scanf from file)
fgetc()	Reads a single character from a file	fputc()	Writes a single character to a file
fgets()	Reads a line/string from file; safer than fgetc loop	fputs()	Writes a string to a file without format specifiers
fread()	Reads a block of raw binary data from file	fwrite()	Writes a block of raw binary data to file
feof()	Returns non-zero if end-of-file indicator is set	rewind()	Moves file pointer back to beginning of file
EOF	End-Of-File marker; typically -1; signals no more data	NULL	Zero pointer; returned by fopen() on failure
"r" mode	Open for read only; file must exist	"w" mode	Open for write; creates file; overwrites if exists
"a" mode	Open for append; adds data at end; creates if needed	Binary Mode	File access without any character translation ("rb", "wb")
Text Mode	File access with newline translation (platform-specific)	Modularity	Dividing a program into independent, interchangeable functions



★ ALL CRITICAL SYNTAX AT A GLANCE

```
// — FUNCTION DECLARATION (Prototype) ————— return_type function_name(param_type1,
param_type2); // — FUNCTION DEFINITION ————— return_type
function_name(type1 p1, type2 p2) { /* body */ return value; } // — FUNCTION CALL
————— result = function_name(arg1, arg2); // — POINTER PARAMETER
(Call by Reference) ————— void modify(int *p) { (*p)++; } modify(&x); // pass address //
— ARRAY PARAMETER ————— void processArr(int arr[], int n); // same
as int *arr processArr(myArray, size); // pass array name = base addr // — FILE OPEN
————— FILE *fp = fopen("filename.txt", "mode"); if(fp =
NULL) { printf("Error!\n"); return 1; } // — FILE WRITE
————— fprintf(fp, "%d %s\n", intVar, strVar); fputc(ch, fp);
fputs(str, fp); fwrite(&data, sizeof(data), 1, fp); // binary // — FILE READ
————— fscanf(fp, "%d %s", &intVar, strVar); ch = fgetc(fp);
fgets(str, 100, fp); fread(&data, sizeof(data), 1, fp); // binary // — FILE CLOSE
————— fclose(fp);
```



PATTERN 1: FUNCTION MODULARIZATION

```
// Decompose large task into functions void
getInput(int *a, int *b); int compute(int a, int
b); void display(int result); int main() { int x,
y, r; getInput(&x, &y); r = compute(x, y);
display(r); }
```

PATTERN 2: ARRAY PROCESSING

```
int findMax(int a[], int n) { int max = a[0];
for(int i=1; i<n; i++) if(a[i] > max) max = a[i];
return max; }
```

PATTERN 3: FILE READ/WRITE LOOP

```
// Write loop for(int i=0; i<n; i++) fprintf(fp,
"%d\n", arr[i]); // Read loop (check EOF) int val;
while(fscanf(fp, "%d", &val) != EOF) {
printf("%d\n", val); }
```

PATTERN 4: POINTER SWAP

```
void swap(int *a, int *b) { int temp = *a; *a =
*b; *b = temp; } // In main: swap(&x, &y);
```


Q: What is a function?

A: A named, reusable block of code that performs a specific task

Q: What does void mean as return type?

A: The function returns no value

Q: What is Call by Value?

A: Passing a copy of the variable; original is not modified

Q: How to swap correctly in C?

A: Use Call by Reference — pass addresses &x, &y to swap function with int *a, *b

Q: How are arrays passed to functions?

A: As base address (pointer); entire array is NOT copied

Q: Scope of global variable?

A: Entire program; accessible from any function

Q: Default value of global variable?

A: 0 (automatically initialized to zero)

Q: What is a File in C?

A: Collection of related data stored permanently on disk storage

Q: What does fopen() return on failure?

A: NULL — always check this before proceeding!

Q: What is "a" mode?

A: Opens file for appending; adds data at end without deleting old content

Q: What does fclose() do?

A: Closes file, flushes buffer to disk, releases file pointer

Q: fprintf vs printf?

A: fprintf writes to a FILE; printf writes to screen (stdout)

Q: What does fgetc() do?

A: Reads one character from file; returns EOF at end

Q: What is a function prototype?

A: Declaration of function before definition; tells compiler name, return type, param types

Q: Formal vs actual parameter?

A: Formal = in definition; Actual = in function call (the real values)

Q: What is Call by Reference?

A: Passing address of variable; original can be modified via pointer

Q: What does (*p)++ do?

A: Increments the VALUE at address p (not the pointer itself)

Q: Scope of local variable?

A: Only within the function/block where it is declared

Q: What is a static variable?

A: Retains its value between function calls; initialized only once

Q: Default value of local variable?

A: Garbage (undefined; not initialized automatically)

Q: What is FILE *fp?

A: Pointer to FILE structure; used to perform all file operations

Q: What is "w" mode?

A: Opens file for writing; creates new file; overwrites if file exists

Q: What is "r" mode?

A: Opens existing file for reading only; returns NULL if not found

Q: What is EOF?

A: End-Of-File marker (value -1); signals no more data in file

Q: fscanf vs scanf?

A: fscanf reads from a FILE; scanf reads from keyboard (stdin)

Q: What does fgets() do?

A: Reads a line (up to n-1 chars) from file into a string buffer

Q: Difference: fwrite vs fprintf?

A: fwrite = binary (raw bytes, faster); fprintf = text (formatted, readable)

Q: What does rewind() do?

A: Resets file pointer to the beginning of the file

Q: How to copy a file in C?

A: Open src in "r", dst in "w"; read char-by-char with fgetc, write with fputc

Q: What is feof()?

A: Returns non-zero if end-of-file indicator is set for the file stream



18

Exam Tips & Common Mistakes

✗ Common Mistakes to AVOID

- Missing function prototype before main()
- Wrong return type (returning int from void function)
- Confusing call by value / reference — value won't swap!
- Forgetting & when passing to pointer parameter
- Writing *p++ instead of (*p)++
- Not checking if fopen() returns NULL
- Forgetting to call fclose() — data loss!
- Opening with "w" when you want append ("a")
- Using local variable after function returns
- Not passing size when passing array to function

✓ Writing Tips for Exams

- Always write function prototype before main()
- Draw memory diagram for pointer questions
- Show call stack for function call tracing
- Use correct file mode ("r"/"w"/"a") with justification
- Always add NULL check after fopen()
- Use while(fscanf()!=EOF) for reading loops
- For swap: explicitly mention call by reference with address
- Mention "static retains value between calls" for static Q
- Label diagrams clearly (address, value, pointer arrow)
- Write output/trace step-by-step for marks

QUICK MEMORY TRICKS**📄 SWAP TRICK**

Value = Copy = Fail
Reference = Address = Works
& to pass, * to receive

📁 FILE TRICK

read = file must exist
write = wipes old content
append = adds to end

📄 VARIABLE TRICK

Local = Stack = Gone when returns
Global = Always alive
Static = Local but remembers



19

Quick Revision Sheet — Last Day Cram

— **FUNCTIONS** — • Syntax: return_type fname(params); //
prototype return_type fname(params) { } // definition result = fname(args); // call • 4 Types: (1)No arg, No ret (2)Arg, No ret (3)No arg, Ret (4)Arg+Ret • Call by Value → copy passed → original UNCHANGED → f(x) • Call by Ref → address passed → original CHANGES → f(&x), param: int *p • Swap: MUST use Call by Reference! • (*p)++ → increment value | *p++ → increment pointer (WRONG!) • Arrays → passed as base pointer automatically → changes affect original • Local variable: stack, limited scope, garbage default • Global variable: everywhere, whole program life, default = 0 • Static variable: local scope BUT retains value between calls — **FILE HANDLING**
— • Open: FILE *fp = fopen("file.txt", "mode"); • MUST CHECK: if(fp == NULL) { exit(1); } • Close: fclose(fp); ← ALWAYS! • Modes: "r" = read only (file must exist) "w" = write (creates/overwrites) "a" = append (adds to end) "rb", "wb", "ab" = binary versions • Write: fprintf(fp, "%d", x); fputc(ch, fp); fputs(str, fp); • Read: fscanf(fp, "%d", &x); fgetc(fp); fgets(buf, n, fp); • Binary: fwrite(&data, sizeof(data), 1, fp); fread(&data, sizeof(data), 1, fp); • EOF check: while(fscanf(fp, "%d", &v) != EOF) { ... } • Char loop: while((ch=fgetc(fp)) != EOF) { ... } — **KEY DIFFERENCES**
— • Value vs Ref: copy vs address | safe vs modifiable | f(x) vs f(&x) • Local vs Global: block scope vs program scope | stack vs global seg • Text vs Binary: readable vs raw | fprintf vs fwrite | slower vs faster • fprintf vs fwrite: formatted text vs raw bytes • fscanf vs fread: formatted read vs raw block read

